



CAPÍTULO 9

Tipos estruturados

Como agrupar valores de tipos diferentes numa única unidade — e manipular, alocar e organizar essas unidades em C.

point2d	
x	float
y	float

O que vamos ver

01 Estruturas

agrupar campos, acessar com `.` ou `->`, sempre passar por ponteiro

02 Definição de novos tipos

`typedef` como apelido — `Point2D` em vez de `struct point2d`

03 Aninhamento de estruturas

uma struct guardando um ponteiro para outra struct

04 Vetores, union e enum

várias structs num só vetor — e dois tipos primos, com papéis mais específicos

Campos de tipos diferentes, uma unidade só

`typedef struct { ... } Time;` agrupa um `int`, uma `string` e outro `int` sob um único tipo.

Time	
id	<code>int</code>
nome	<code>char[50]</code>
pontos	<code>int</code>

A seta é só um atalho

p é um ponteiro para uma struct com um campo
x — as duas formas abaixo são idênticas.

`(*p).x`

desreferencia p,
depois acessa o
campo

≡

`p->x`

mesma coisa,
escrita de um jeito
mais direto

Uma struct pode viver na pilha ou no heap

`malloc(sizeof(...))` tira a struct da pilha e a coloca no heap, com vida própria.

```
struct point2d point;
```

pilha – existe
enquanto a função
existir

VS

```
malloc(sizeof(struct point2d))
```

heap – existe até
um free explícito

typedef: um apelido para o tipo

Point2D e struct point2d são exatamente o mesmo tipo — só muda o que se escreve.

Sem typedef

```
struct point2d p;  
struct point2d *ptr;
```

Com typedef

```
Point2D p;  
Point2D *ptr;
```

Uma struct guardando um ponteiro para outra

circle->center->x encadeia duas setas: primeiro chega ao Point2D, depois ao campo x dele.

Circle	
center	Point2D *
radius	double

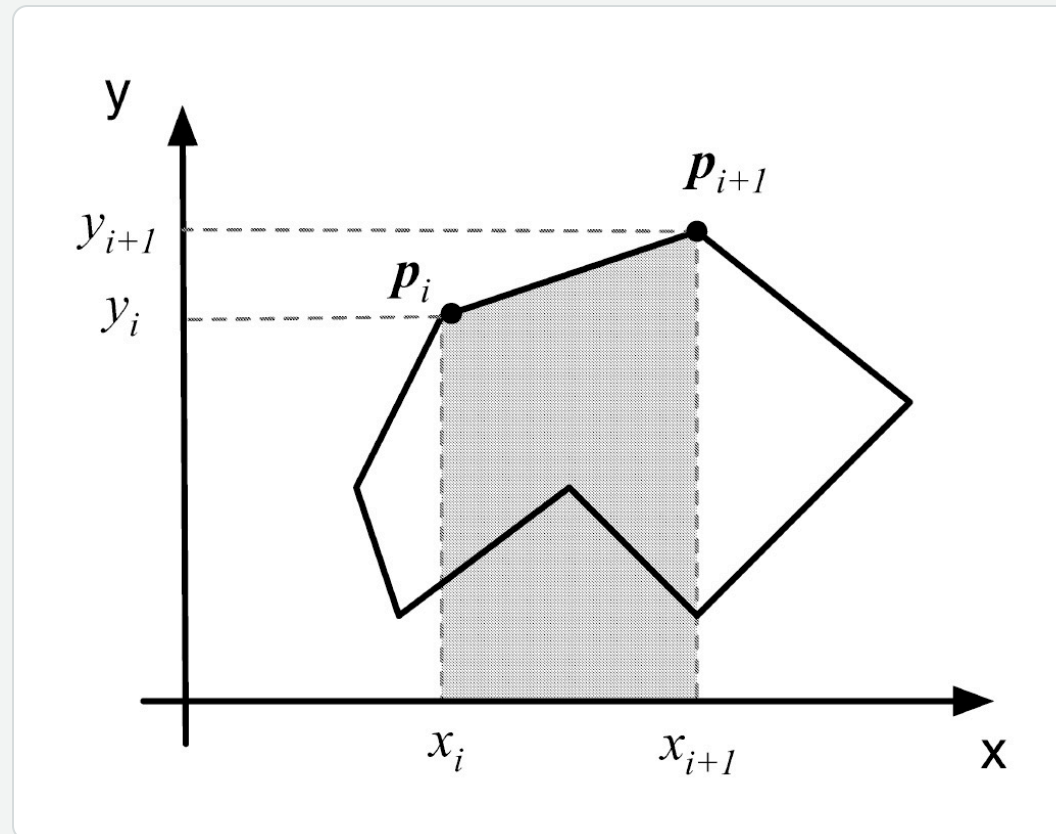


center

Point2D	
x	double
y	double

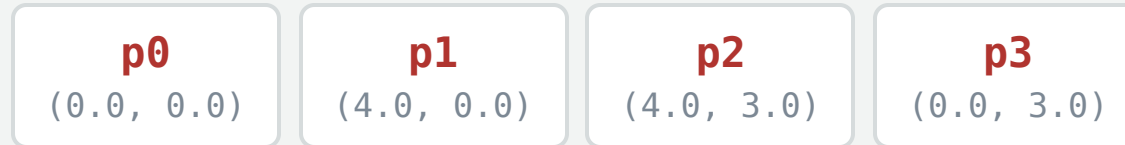
Um polígono é só um vetor de pontos

Cada vértice é um Point2D; o polígono inteiro é um vetor dessas structs, na ordem dos vértices.



Point2D poligono[4] — quatro vértices, um vetor só

Cada posição já reserva espaço para um Point2D completo — sem vetores separados de x e de y.



06_UNION_ENUM_VECTOR/MAIN.C

```
Point2D poligono[4] = {  
    {0.0, 0.0}, {4.0, 0.0},  
    {4.0, 3.0}, {0.0, 3.0}  
};  
  
for (int i = 0; i < 4; i++)  
    printf("p%d = (%.1f, %.1f)\n",  
        i, poligono[i].x, poligono[i].y);
```

Campos separados vs. espaço compartilhado

struct

Uma struct reserva espaço para cada campo.
uma union reserva um único espaço, do tamanho do maior campo.

int i

4 bytes

float f

4 bytes

campos coexistem — 8
bytes ao todo

union

int i
float f

4 bytes (compartilhados)

escrever num campo
sobrescreve o outro

```
06_UNION_ENUM_VECTOR/MAIN.C
```

```
typedef union  
{  
    int como_inteiro;  
    float como_real;
```

Nomes em vez de números soltos

Cada nome do enum é, por baixo, uma constante inteira sequencial, começando em 0.

DOM	SEG	TER	QUA	QUI	SEX	SAB
0	1	2	3	4	5	6

```
06_UNION_ENUM_VECTOR/MAIN.C
typedef enum
{
    TIPO_INTEIRO,
    TIPO_REAL
} TipoValor;

TipoValor tipo = TIPO_REAL;
// TIPO_INTEIRO = 0, TIPO_REAL = 1
if (tipo == TIPO_INTEIRO)
    ...
```

AGORA É COM VOCÊ

Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada. Depois, teste o que aprendeu no quiz.



Notas de Aula e Exemplos thiagopx.github.io/ed/07_structs